## Phase 1 Exercise Overview

1. Authors:

   Group z4

   https://github.com/mGarbowski/tbd-workshop-1

2. Follow all steps in README.md.
3. From available Github Actions select and run destroy on master branch.
4. Create new git branch and:
   1. Modify tasks-phase1.md file.
   2. Create PR from this branch to **YOUR** master and merge it to make new release.

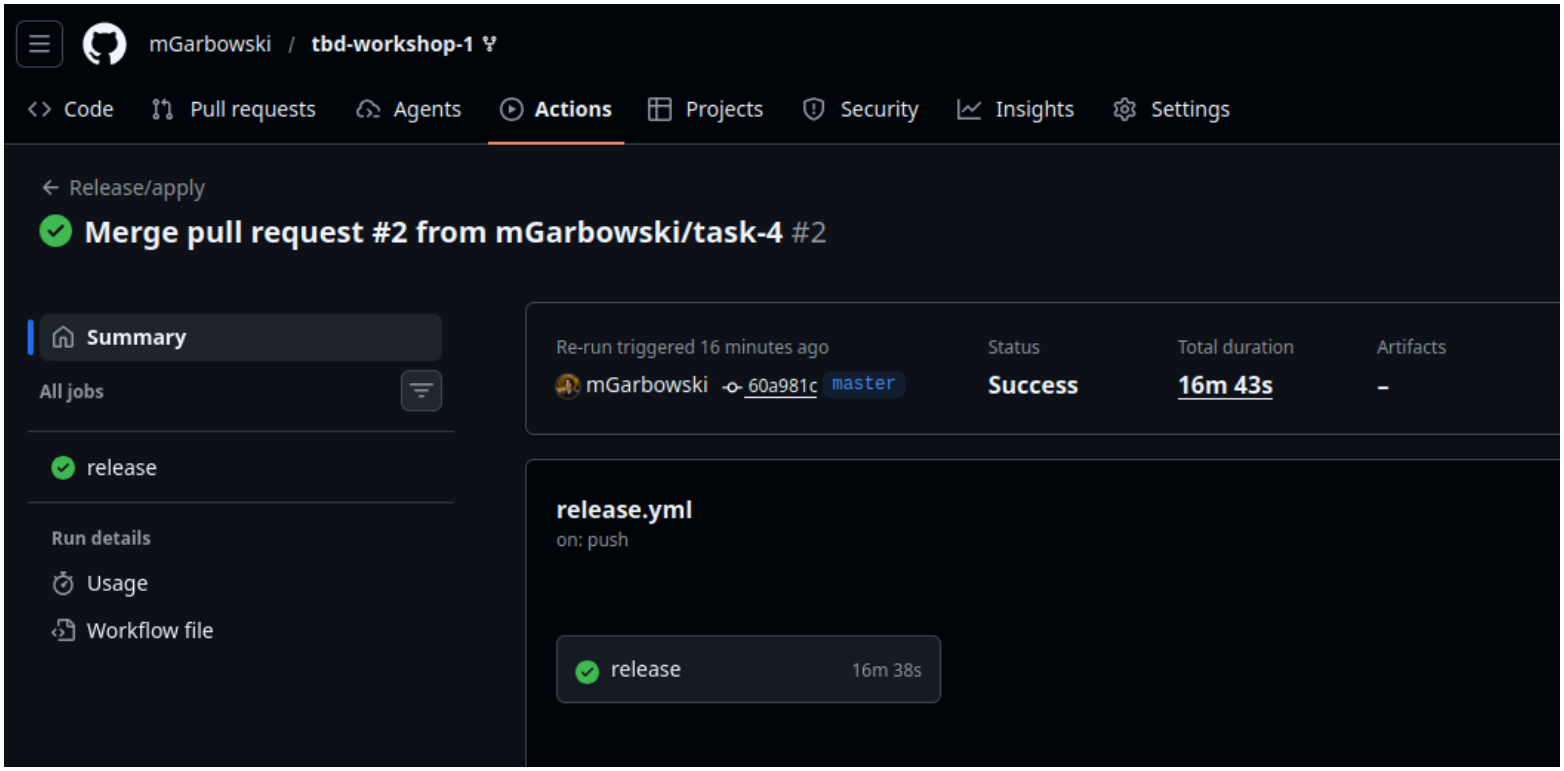


Figure 1: GitHub Actions screenshot

5. Analyze terraform code. Play with terraform plan, terraform graph to investigate different modules.

   Module airflow configures an account with IAM roles and configures a Kubernetes cluster with 2 nodes where Airflow will be deployed

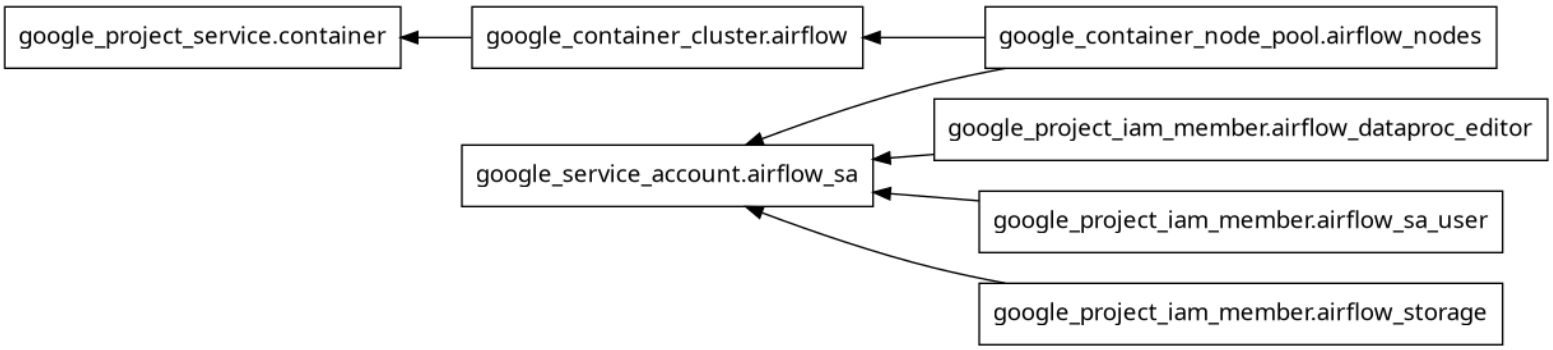


Figure 2: Terraform graph for airflow module

6. Reach YARN UI

   Hint: the Dataproc cluster has `internal_ip_only = true`, so you need to use an IAP tunnel. See: `gcloud compute ssh` with `-- -L <local_port>:localhost:<remote_port>` and `--tunnel-through-iap` flag. YARN ResourceManager UI runs on port **8088**.

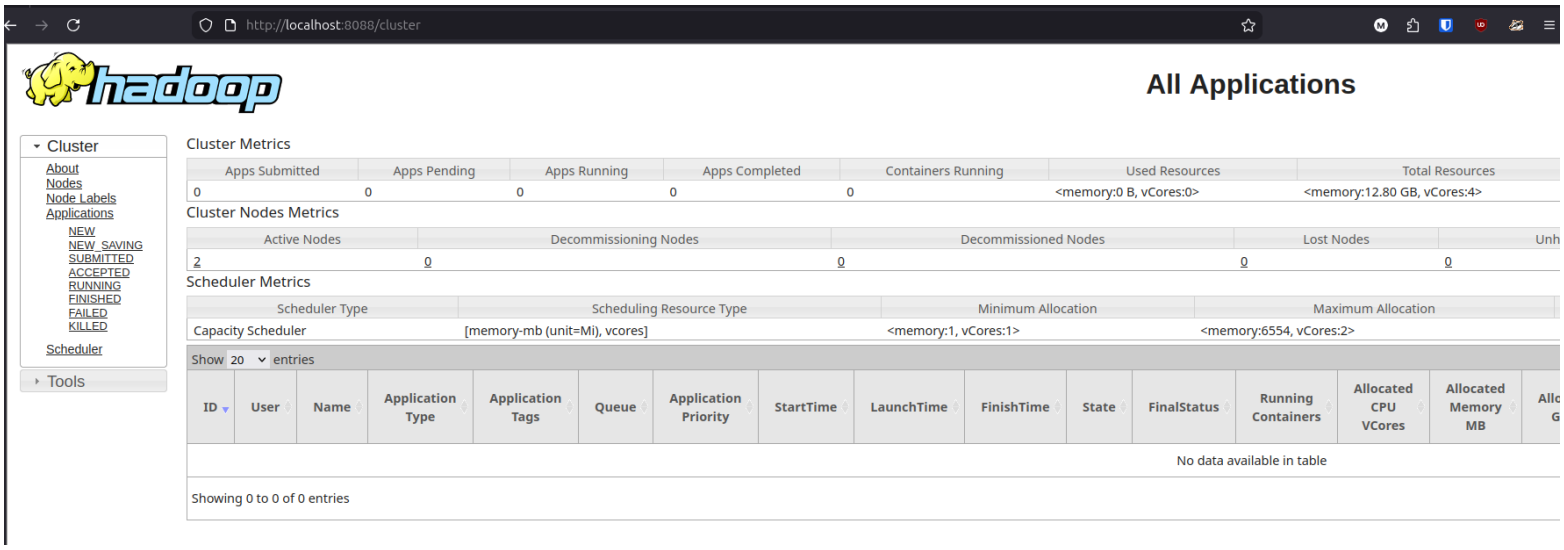


Figure 3: Yarn UI screenshot

Command to setup SSH tunnel using IAP

```
gcloud compute ssh tbd-cluster-m --project=tbd-2026l-325157 --zone=europe-west1-b --tunnel-through-iap -- -L 8
```

To check node name and zone

```
    tbd-workshop-1 git:(task-5) gcloud compute instances list --project=tbd-2026l-325157
NAME                                          ZONE            MACHINE_TYPE   PREEMPTIBLE  INTERNAL_IP  EXTER
gke-airflow-cluster-airflow-pool-ede3a231-89w6  europe-west1-b  e2-standard-2               10.10.10.5   34.76
gke-airflow-cluster-airflow-pool-ede3a231-ccdf  europe-west1-b  e2-standard-2               10.10.10.4   104.1
tbd-cluster-m                                 europe-west1-b  e2-standard-2               10.10.10.7
tbd-cluster-w-0                               europe-west1-b  e2-standard-2               10.10.10.6
tbd-cluster-w-1                               europe-west1-b  e2-standard-2               10.10.10.8
```

7. Draw an architecture diagram (e.g. in draw.io) that includes:
   1. Description of the components of service accounts
   2. List of buckets for disposal

   Buckets `bash gcloud storage buckets list --format="table(name, location)" gcloud iam service-accounts list`

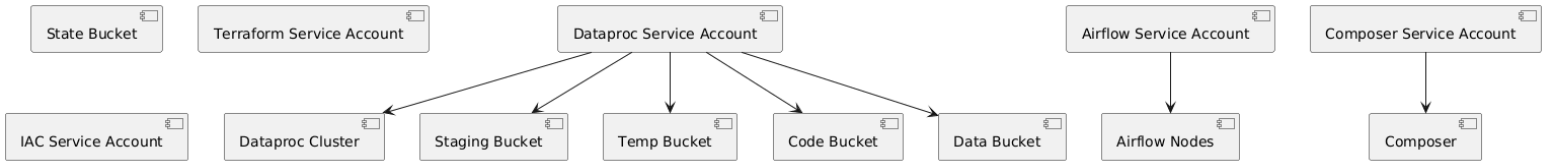


Figure 4: Component diagram with service accounts and buckets

8. Create a new PR and add costs by entering the expected consumption into Infracost For all the resources of type: `google_artifact_registry_repository`, `google_storage_bucket` create a sample usage profiles and add it to the Infracost task in CI/CD pipeline. Usage file example

```yaml
version: 0.1
resource_type_default_usage:
google_artifact_registry_repository:
    storage_gb: 10

google_storage_bucket:
    storage_gb: 100
```

9. Find and correct the error in spark-job.py

   After `terraform apply` completes, connect to the Airflow cluster: `bash gcloud container clusters get-credentials airflow-cluster --zone europe-west1-b --project PROJECT_NAME`

   Then check the external IP (AIRFLOW_EXTERNAL_IP) of the webserver service: kubectl get svc -n airflow airflow-webserver

**github-actions** bot commented 19 minutes ago • edited

💰 **Infracost report**

**Monthly estimate increased by $14** 📈

| Changed project | Baseline cost | Usage cost* | Total change | New monthly cost |
|---|---|---|---|---|
| mGarbowski/tbd-workshop-1 | +$0 | +$9 | +$9 (+5%) | $194 |
| mGarbowski/tbd-workshop-1/bootstrap | +$0 | +$2 | +$2 | $2 |
| mGarbowski/tbd-workshop-1/mlops | +$0 | +$3 | +$3 (+10%) | $30 |

*Usage costs were estimated using infracost-usage.yml, see docs for other options.

▸ Estimate details (includes details of unsupported resources)

This comment will be updated when code changes.

Figure 5: Infracost gitHub PR comment

Note: If EXTERNAL-IP shows , wait a moment and retry — LoadBalancer IP allocation may take 1-2 minutes.

DAG files are synced automatically from your GitHub repo via git-sync sidecar. Airflow variables and the `google_cloud_default` GCP connection are also configured by Terraform.

a) In the Airflow UI (http://AIRFLOW_EXTERNAL_IP:8080, login: admin/admin), find the `dataproc_job` DAG, unpause it and trigger it manually.

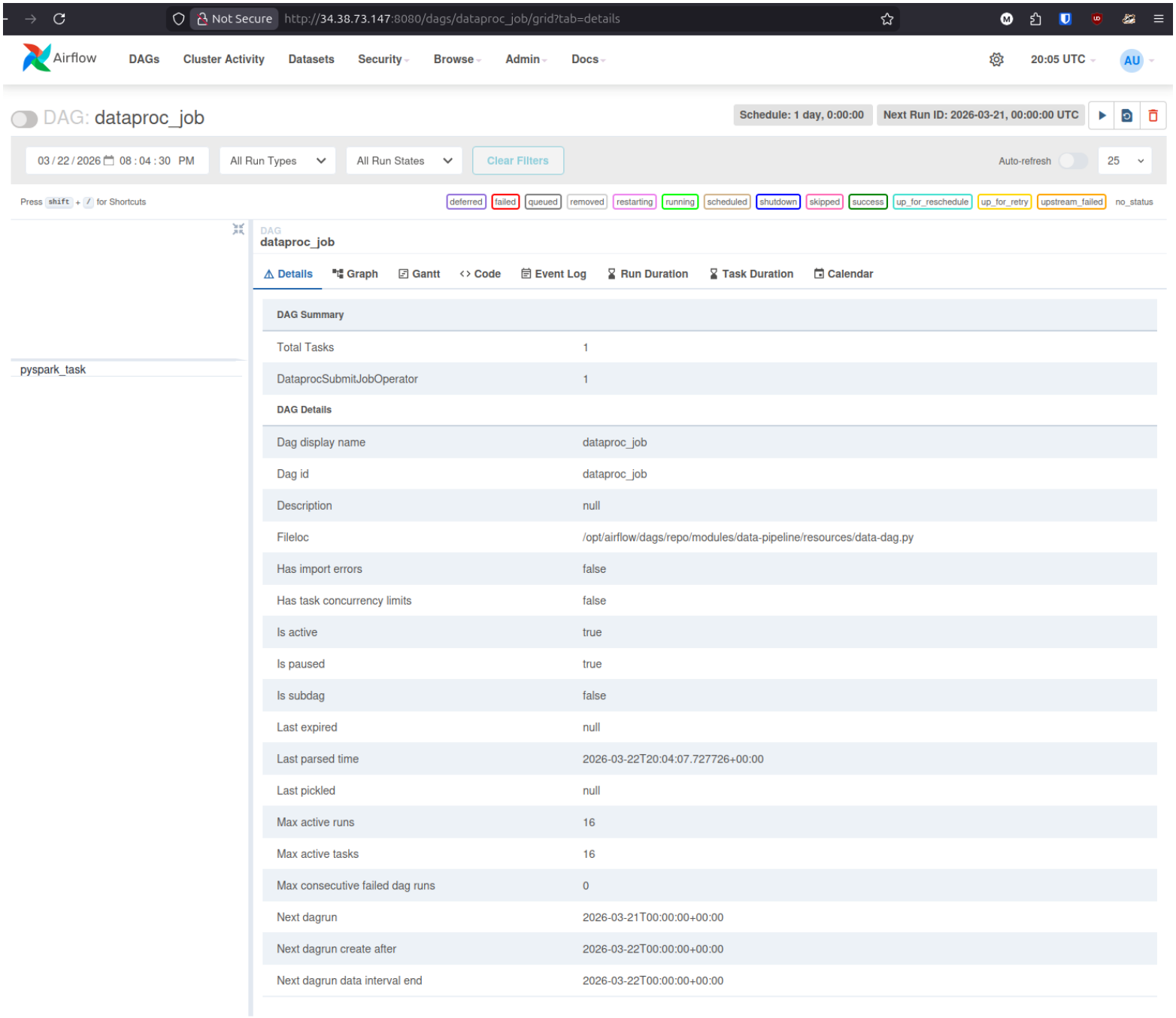


Figure 6: Airflow UI screenshot

b) The DAG will fail. Examine the task logs in the Airflow UI to find the root cause.

Airflow logs:

```
status {
state: ERROR
details: "Google Cloud Dataproc Agent reports job failure. If logs are available, they can be found at:\nhttps
state_start_time {
    seconds: 1774210120
    nanos: 128620000
}
}
```

GCP logs in the link from above:

```
py4j.protocol.Py4JJavaError: An error occurred while calling o96.orc.
: com.google.cloud.hadoop.repackaged.gcs.com.google.api.client.googleapis.json.GoogleJsonResponseException: 4(
POST https://storage.googleapis.com/upload/storage/v1/b/tbd-2026l-9010-data/o?ifGenerationMatch=0&uploadType=m
{
"code": 404,
"errors": [
    {
    "domain": "global",
    "message": "The specified bucket does not exist.",
    "reason": "notFound"
    }
],
"message": "The specified bucket does not exist."
}
```

The `spark-job.py` has a hardcoded storage bucket name, there is a comment saying it must be changed to my own bucket.

c) Fix the error in `modules/data-pipeline/resources/spark-job.py` and re-upload the file to GCS:

```
gsutil cp modules/data-pipeline/resources/spark-job.py gs://PROJECT_NAME-code/spark-job.py
```

Then trigger the DAG again from the Airflow UI.

gs://tbd-2026l-325157-code/spark-job.py

d) Verify the DAG completes successfully and check that ORC files were written to the data bucket:

```
gsutil ls gs://PROJECT_NAME-data/data/shakespeare/
```

10. Create a BigQuery dataset and an external table using SQL

Using the ORC data produced by the Spark job in task 9, create a BigQuery dataset and an external table.

Note: the dataset must be created in the same region as the GCS bucket (`europe-west1`), e.g.:

```
bq mk --dataset --location=europe-west1 shakespeare
```

Executed in the web GCP console:

```
CREATE EXTERNAL TABLE `shakespeare.orc_table`
OPTIONS (
    format = 'ORC',
    uris = ['gs://tbd-2026l-325157-data/data/shakespeare/*.orc']
);
```

```
SELECT word, sum_word_count
FROM shakespeare.orc_table
ORDER BY sum_word_count DESC
LIMIT 10;
```

```
word    sum_word_count
the 25568
I   21028
and 19649
to  17361
of  16438
a   13409
you 12527
my  11291
in  10589
is  8735
```

Why does ORC not require a table schema: as per docs

> ORC is a **self-describing** type-aware columnar file format designed for Hadoop workloads.

11. Add support for preemptible/spot instances in a Dataproc cluster

```
[modules/dataproc/main.tf](https://github.com/mGarbowski/tbd-workshop-1/blob/master/modules/dataproc/main.tf)

```tf
resource "google_dataproc_cluster" "tbd-dataproc-cluster" {
```
```

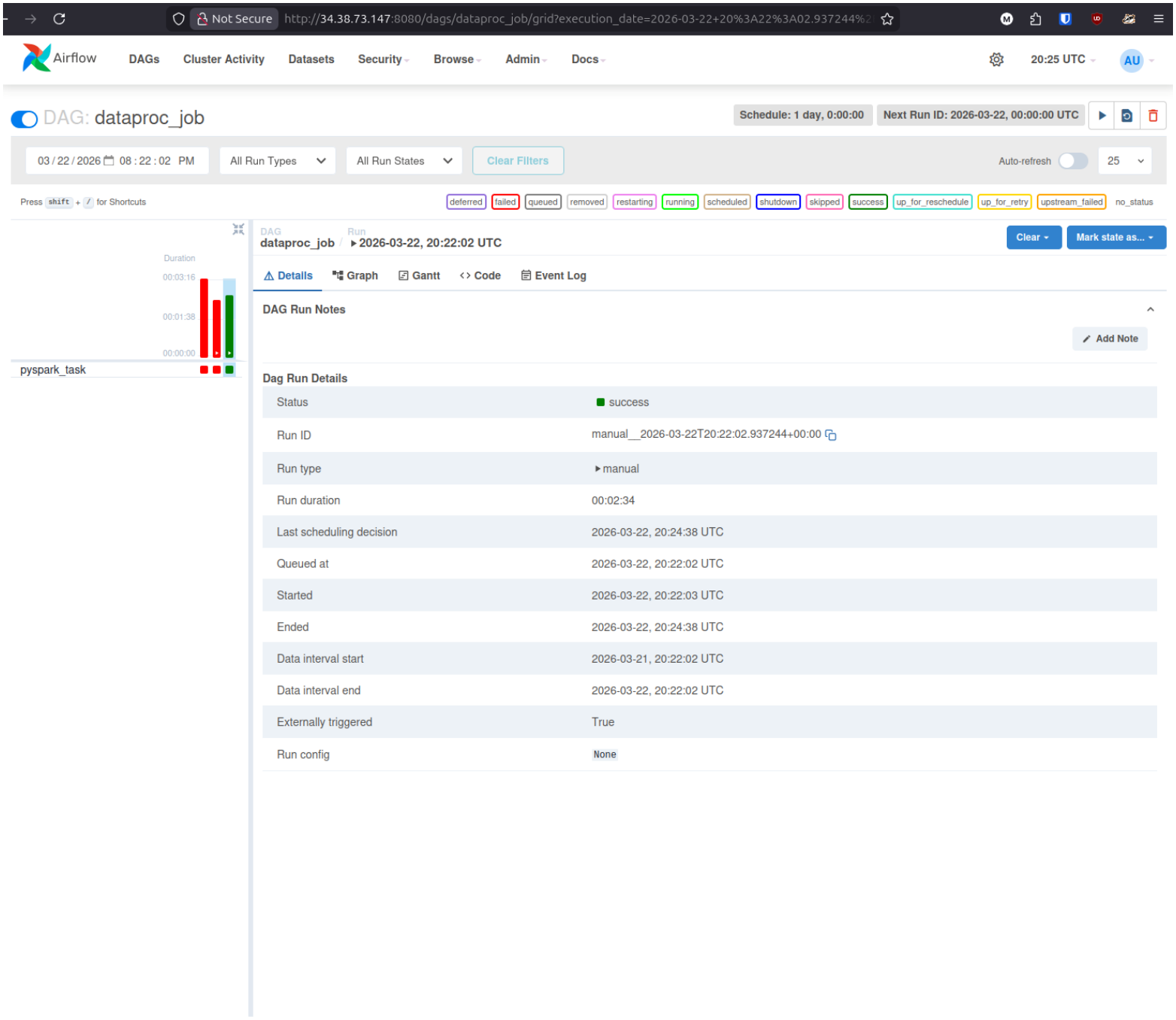


Figure 7: Airflow successful DAG run

```
    ...

    cluster_config {
        ...
        gce_cluster_config {
            ...

            worker_config {
                num_instances = 0
                machine_type  = var.machine_type
            }

            preemptible_worker_config {
                num_instances = 2
                preemptibility = "PREEMPTIBLE"
                disk_config {
                    boot_disk_type    = "pd-standard"
                    boot_disk_size_gb = 100
                }
            }
        }
    }
}
```

12. Triggered Terraform Destroy on Schedule or After PR Merge. Goal: make sure we never forget to clean up resources and burn money.

Add a new GitHub Actions workflow that: 1. runs terraform destroy -auto-approve 2. triggers automatically:

a) on a fixed schedule (e.g. every day at 20:00 UTC)

b) when a PR is merged to master containing [CLEANUP] tag in title

Steps: 1. Create file .github/workflows/auto-destroy.yml 2. Configure it to authenticate and destroy Terraform resources 3. Test the trigger (schedule or cleanup-tagged PR)

Hint: use the existing `.github/workflows/destroy.yml` as a starting point.

```yaml
name: Auto Destroy
description: Automatically destroy GCP infrastructure on schedule or on merging PR to master that contains "[CLEANU
on:
  schedule:
    - cron: '0 0 * * *' # Every day at midnight
  pull_request:
    types:
      - closed
    branches:
      - master


permissions: read-all
jobs:
  destroy-release:
    if: github.event.pull_request.merged == true && contains(github.event.pull_request.title, '[CLEANUP]')
    #.... same as destroy.yml
```

A workflow like this that runs on cron schedule could prevent unwanted costs if I were to forget to manually run the destroy workflow. Then, the cloud infrastructure would still be online and generating costs.

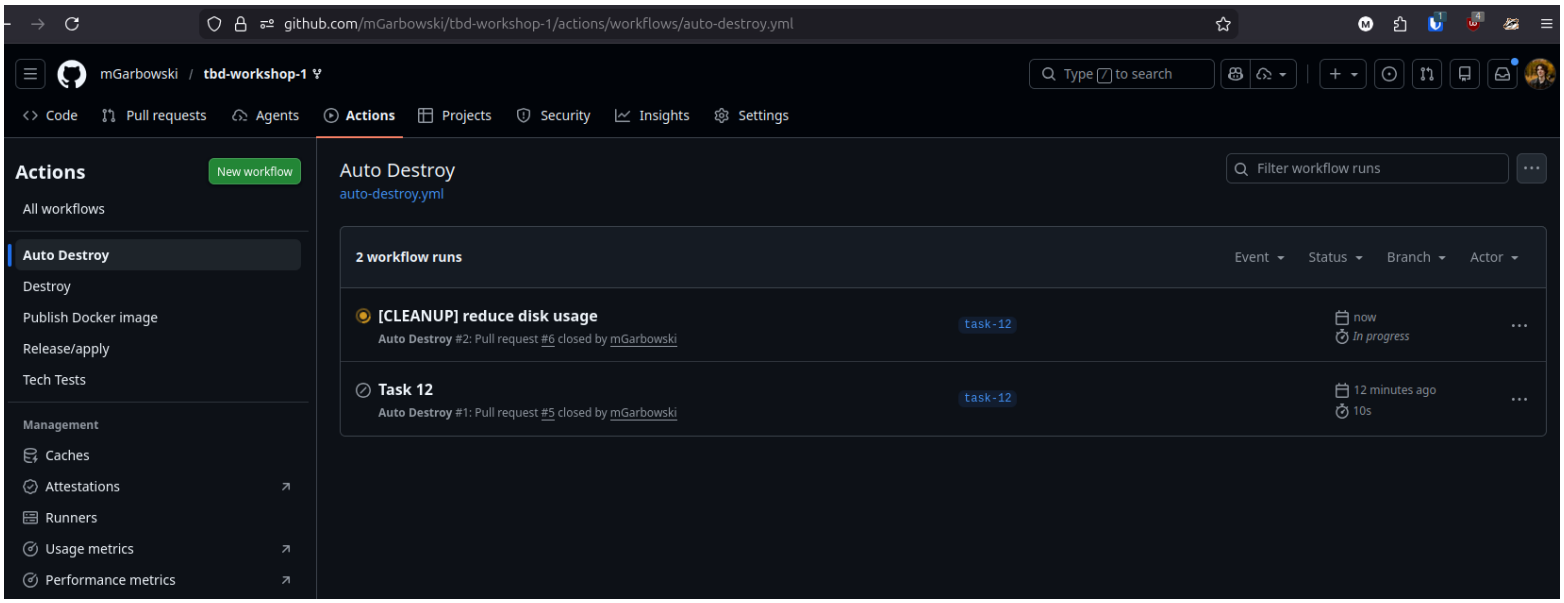


Figure 8: GitHub Actions screenshot